Module 05 — Editing one file safely

Claude Code 101 · Beginner Workshop · Module 5 of 8

This is the first lesson where Claude actually changes a file. We do it in a Git repo so every change is reversible.

What you'll learn

By the end of this 30-minute lesson you will be able to:

1. Ask Claude to edit one file and read the proposed diff before accepting.
2. Accept a hunk, reject a hunk, or undo the whole change with `git restore`.
3. Make a habit out of committing **before** you let Claude edit.

Why this matters

- The single biggest beginner accident is "Claude changed my file and I don't know what it did". The fix is one Git command — but only if you committed first.
- Real engineering speed comes from being able to say "try it" without fear. Git is what makes "try it" cheap.
- The "read the diff before you accept" habit is the cheapest review process you'll ever have. It catches most mistakes for free.

The one concept

> **Commit first → ask for the edit → read the diff → accept, reject, or `git restore`.**

That sentence is the entire safety net. Memorize it.

- A **diff** shows you exactly which lines Claude wants to add (+) and remove (–).
- A **hunk** is one contiguous block of changes inside that diff. You can accept hunk by hunk; you don't have to take everything.
- `git restore <file>` returns the file to the last committed state. As long as you committed first, nothing is ever lost.

Show me

A safe edit, start to finish:

```
$ git status
On branch main
nothing to commit, working tree clean

$ claude
> Open hello.py. Add a docstring to the greet() function. Do not change behaviour.

Proposed change to hello.py:

 def greet(name):
+    """Return a friendly greeting for the given name."""
     return f"Hello, {name}!"

Accept? (y/n)
> y

Applied. 1 file changed.
> /exit

$ git diff
... shows the same 1-line addition ...
```

Try it yourself

A tiny Git repo lives at `exercises/beginner/part-05/starter/`. Your job: ask Claude to add a docstring to `greet()`, accept the diff, then run `git restore` to undo it on purpose. You will end this lesson knowing the file is exactly as it started.

Time budget: 15 minutes (mostly Git ceremony, not Claude).

Common mistakes

- **Editing without committing first.** `git restore` only works back to the last commit. No commit = no undo.
- **Accepting a diff you didn't read.** A multi-hunk diff often contains one good change and one surprise.
- **Editing files outside a Git repo.** Don't. Use Git for anything you would not want to retype.
- **Mixing two unrelated edits.** Ask for one change at a time. One diff, one decision, one commit.

Lesson reflection

Take 90 seconds:

1. When you read the diff, was the change exactly what you asked for, or did Claude do something extra?
2. After `git restore`, is your file identical to the committed version? Verify with `git diff` (should be empty).
3. If you had been working on three different files and Claude edited all three, would you have known how to undo just one?

What's next

Module 06 — **CLAUDE.md cheat sheet** — teaches you to give Claude a tiny file of project context so it stops asking the same questions every session.

Budget for Module 06: 25 minutes.

Glossary card

- **Diff**: The set of lines Claude proposes to add and remove from a file.
- **git restore**: A Git command that undoes uncommitted changes to a file.
- **Hunk**: One contiguous chunk of changed lines inside a larger diff.
- **Reversible edit**: A change you can undo with one command.